

Drone_Swarm_Simulator_v2:

техническое описание симулятора роя и цикла эксперимента

По исходному коду репозитория `Kursov3/Drone_Swarm_Simulator_v2` и SITL Ardu

Аннотация

Документ описывает назначение, архитектуру и основные алгоритмы проекта `Drone_Swarm_Simulator_v2`: единый процесс Python, подключающийся по MAVLink к нескольким экземплярам ArduPilot SITL (и опционально к Webots), обмен координатами внутри процесса, регуляторы PID и сценарии `leader_forward_back`, `linear_chain2`. Явно подчёркивается, что жёсткотельная динамика и интегрирование движения выполняются в SITL/Webots, а не в коде Python данного репозитория. При первом появлении поясняются обозначения в формулах.

Содержание

1	Назначение проекта и структура каталогов	2
2	Обмен данными: частоты и механизмы	2
3	Жизненный цикл эксперимента	3
4	Связи, топология и общая система координат	4
5	Регуляторы и законы управления в проекте	5
5.1	Класс <code>PIDRegulator</code> (<code>core/control/pid.py</code>)	5
5.2	Сценарий <code>leader_forward_back</code> : последователь и ошибки	5
5.3	Сценарий <code>linear_chain2</code> : конечные точки, NED→корпус, закон Anatoliy	5
6	Физика и динамика полёта: ответственность SITL и Webots	7
6.1	Роль кода Python в данном репозитории	7
6.2	Мультикоптер в SITL: иерархия классов и силы	7
6.3	Дискретное обновление в <code>Aircraft::update_dynamics</code>	8
6.4	Параметр <code>SCHED_LOOP_RATE</code>	9
6.5	Режим Webots (<code>SIM_Webots_Python.cpp</code>)	9
7	Заключение	9

1 Назначение проекта и структура каталогов

Назначение. Проект предназначен для исследования алгоритмов координации роя однотипных мультикоптеров в связке *ArduPilot SITL + MAVLink + Python*. Управляющий код на Python задаёт высокоуровневые команды (режимы полёта, RC_OVERRIDE, запросы потоков телеметрии), считывает локальные координаты и ориентацию с каждого борта, распространяет информацию о положениях между логическими «агентами» в одном адресном пространстве процесса и записывает результаты в CSV; предусмотрено воспроизведение в ROS/RViz и опциональная 2D-визуализация.

Структура (по README.md).

- `launch_simulation.py` — единая точка входа для одиночного запуска (SITL и/или Webots, сценарий, опции визуализатора).
- `run_batch.py` — пакетный запуск серии экспериментов.
- `config/` — файлы параметров ArduPilot (в т.ч. `iris.parm`).
- `core/` — ядро: MAVLink-обёртка, мониторы координат и скорости, PID, логирование.
- `scenarios/` — сценарии экспериментов (`leader_forward_back.py`, `linear_chain2.py` и др.).
- `replay/` — воспроизведение логов в ROS/RViz.
- `visualizer/` — 2D-визуализация (matplotlib), приём позиций по UDP.
- `experiments/`, `docs/` — результаты и локальная документация (не обязательны для работы кода).

2 Обмен данными: частоты и механизмы

Все величины времени ниже — в секундах (с), частоты — в герцах (Гц), если не указано иное.

Аргумент лаунчера `-exchange-hz`. В `launch_simulation.py` параметр командной строки `-exchange-hz` имеет значение по умолчанию $f_{\text{exch}} = 50$ Гц и передаётся запускаемому сценарию как `-exchange-hz`, задавая номинальную частоту цикла обмена координатами между контроллерами дронов в одном процессе Python.

Потоки MAVLink: запрос интервала сообщений. Класс `MAVLinkWorker` (`core/mavlink/work`) по командам инициализации отправляет `MAV_CMD_SET_MESSAGE_INTERVAL` для сообщений `LOCAL_POSITION_NED` и `ATTITUDE`. Интервал между экземплярами одного сообщения задаётся как

$$T_{\text{msg}} = \frac{10^6}{f_{\text{stream}}} \mu\text{с}, \quad (1)$$

то есть в коде `interval_us = int(1e6 / hz)` при заданной частоте потока f_{stream} (по умолчанию в шагах инициализации сценариев $f_{\text{stream}} = 50$ Гц). Таким образом, телеметрия положения и угловой ориентации с борта запрашивается с частотой 50 Гц.

Цикл приёма в рабочем потоке MAVLink. В методе `_run_loop` вызывается `recv_match(..., blocking=False, timeout=0.01)`, т.е. тайм-аут опроса входящих сообщений $\tau_{\text{recv}} = 0,01$ с. После попытки чтения выполняется `time.sleep(0.01)` — пауза $\tau_{\text{sleep}} = 0,01$ с между итерациями цикла потока. Итоговая частота опроса не жёстко равна 100 Гц из-за обработки очереди команд, но порядок величины — около десятков герц опроса сокета при номинальной доставке сообщений 50 Гц с автопилота.

Поддержание канала RC (RC_OVERRIDE). В `DroneController.start_rc_heartbeat` (`core/control/drone_controller.py`) фоновый поток с периодом $T_{\text{rc}} = 0,2$ с повторно отправляет последние заданные значения каналов RC_OVERRIDE через `MAVLinkWorker`, чтобы канал управления не считался устаревшим со стороны автопилота.

Визуализатор: UDP JSON. Модуль `visualizer/position_publisher.py` отправляет словарь позиций (и опционально словарь частот) на узел 127.0.0.1, порт 15551 по протоколу UDP; полезная нагрузка — строка JSON (`json.dumps`, кодировка UTF-8). Сценарии вызывают `publish_positions` из цикла обмена координатами при наличии импорта.

3 Жизненный цикл эксперимента

Ниже — типичная последовательность при запуске `python launch_simulation.py`

1. **Старт внешних процессов.** При режиме Webots сначала запускается симулятор Webots с сформированным миром; затем для каждого дрона — `sim_vehicle.py` (ArduCopter, SITL). В режиме только SITL запускаются N процессов `sim_vehicle.py` без Webots.
2. **Ожидание готовности MAVLink.** После старта SITL лаунчер вызывает `wait_all_sitl_heartbeats`: на каждом UDP-порту ожидается сообщение HEARTBEAT с ожидаемым системным идентификатором (тайм-аут по умолчанию 120 с на каждый борт).
3. **Пауза стабилизации (*settle*).** Вычисляется `settle_sec = min(8.0, 2.0 + 0.25 * max(0, num_drones - 4))`; процесс засыпает на эту длительность перед запуском сценария.
4. **Старт сценария.** Подпроцесс Python запускает выбранный скрипт из `scenarios/` с аргументами `-drones`, при необходимости `-duration`, `-experiment-dir`, `-run-id`, `-exchange-hz`.
5. **Инициализация в сценарии.** Для каждого дрона создаётся `DroneController`, вызывается `connect` (создание `MAVLinkWorker`, ожидание первого heartbeat внутри `start`), затем `initialize(init_steps)`: последовательность команд в очереди рабочего потока (режим, арминг, взлёт, переход в удержание позиции, нейтральный RC, запрос потоков LOCAL_POSITION_NED/ATTITUDE на 50 Гц и т.д.). Запускается `start_rc_heartbeat`.

6. **Цикл обмена координатами.** Параллельно или в главном потоке сценария выполняется `coordinate_exchange_loop`: сбор позиций, перевод в общую NED-систему (см. раздел 4), рассылка в `other_drones_positions`, опционально публикация в визуализатор.
7. **Циклы управления.** Потоки или циклы сценария (`follower_loop`, `internal_chain_anatol_endpoint_loop` и т. п.) используют кэшированные позиции соседей и формируют `RC_OVERRIDE`.
8. **Логирование и завершение.** Запись CSV/метаданных (в зависимости от сценария), по завершении сценария или сигналу SIGINT/SIGTERM лаунчер завершает дочерние процессы (SITL, визуализатор, Webots).

Воспроизведение записанных траекторий выполняется отдельными скриптами в `replay/` и не является частью описанного цикла `launch_simulation.py`, но использует те же логические данные (CSV, метаданные).

4 Связи, топология и общая система координат

Один процесс — N экземпляров SITL. Управляющее приложение Python не открывает MAVLink-соединений «дрон-дрон». Для дрона с индексом экземпляра $i \in \{0, \dots, N-1\}$ в `launch_simulation.py` задаётся вывод `sim_vehicle.py` на UDP `127.0.0.1:(14551 + 10 i)`. Системный идентификатор MAVLink (`sysid`) назначается равным $i + 1$. Таким образом, топология обмена с носителями — *звезда*: центральный процесс Python и N независимых UDP-каналов к автопилотам.

Общий кадр NED и смещение по востоку. Каждый SITL получает свою «домашнюю» точку: базовые широта/долгота фиксированы, долгота сдвигается так, чтобы в общей локальной системе NED (ось x — север, y — восток, z — вниз) соседние экземпляры были разнесены по востоку на 2 м: для идентификатора дрона в конфигурации d_{id} (совпадает с `sysid`) поправка к координате y при переходе к общему кадру есть $(d_{id} - 1) \cdot 2$ м (см. функции вроде `_my_position_common` в `linear_chain2.py` и аналогичную логику в `leader_forward_back.py`).

Обмен между дронами внутри процесса. Словарь `other_drones_positions` в `DroneController` обновляется из `coordinate_exchange_loop`: каждый контроллер записывает позиции остальных, уже приведённые к общему NED. Прямой рассылки MAVLink между бортами нет.

Топологии сценариев.

- **Звезда (лидер — последователи).** В `leader_forward_back` последователи используют позицию лидера из `get_other_drones_positions`; граф информационных зависимостей — «лидер $\rightarrow$ каждый последователь».
- **Цепь.** В `linear_chain2` выделяются концевые «якоря» и внутренние звенья; внутренний закон опирается на множество соседей в радиусе видимости вдоль и поперёк отрезка между опорными точками, т. е. информационно релевантна геометрия цепи (соседи по проекции на отрезок), а не полный граф роя.

5 Регуляторы и законы управления в проекте

5.1 Класс PIDRegulator (core/control/pid.py)

Обозначения: $e(t)$ — входная ошибка регулирования; Δt — шаг времени между вызовами (если не передан явно, оценивается по часам); K_p , K_i , K_d — коэффициенты пропорциональной, интегральной и дифференциальной составляющих; $I_{\lim}$ — предел модуля интеграла (`integral_limit`, антиперегрузка интегратора при $I_{\lim} > 0$); $U_{\lim}$ — симметричное ограничение выхода (`output_limit`); $\alpha_d \in [0, 1]$ — параметр сглаживания производной (`derivative_alpha`), при $\alpha_d > 0$ используется рекуррентное низкочастотное фильтрование разностной оценки производной ошибки.

Алгоритм одного шага:

$$I \leftarrow I + e \Delta t, \quad I \leftarrow \text{sat}(I; -I_{\lim}, I_{\lim}) \quad \text{при } I_{\lim} > 0, \quad (2)$$

$$D_{\text{raw}} = \frac{e - e_{\text{prev}}}{\Delta t} \quad (\text{при } \Delta t > 0), \quad (3)$$

$$D \leftarrow \alpha_d D + (1 - \alpha_d) D_{\text{raw}} \quad (\text{при } \alpha_d > 0), \quad (4)$$

$$u = K_p e + K_i I + K_d D, \quad u \leftarrow \text{sat}(u; -U_{\lim}, U_{\lim}), \quad (5)$$

$$e_{\text{prev}} \leftarrow e. \quad (6)$$

Здесь $\text{sat}(x; a, b)$ — отсечение x интервалом $[a, b]$. Выход u далее масштабируется сценарием в PWM.

5.2 Сценарий `leader_forward_back`: последователь и ошибки

Пусть $p^L = (x^L, y^L, z^L)^\top$ — позиция лидера в общем NED, $p^F = (x^F, y^F, z^F)^\top$ — позиция последователя в том же кадре (с учётом смещения по y на $(d_{\text{id}} - 1) \cdot 2$ м из сырой телеметрии). Задаются ошибки

$$e_x = x^L - x^F, \quad e_y = (y^L - y^F) - d^*, \quad (7)$$

где $d^* = 2$ м — номинальный шаг формации вдоль оси востока (`FORMATION_D_STAR_M`, совпадает с поправкой “-2” в коде для выравнивания по ряду). По оси “вперёд/назад” (pitch-канал RC) используется выход PID от e_x , по поперечной оси (roll) — от e_y . Нейтральное значение PWM для стиков $u_{\text{neut}} = 1500$ (`RC_NEUTRAL`). Закон отправки (упрощённо):

$$u_{\text{pitch}} = u_{\text{neut}} - \text{PID}_p(e_x), \quad u_{\text{roll}} = u_{\text{neut}} + \text{PID}_r(e_y), \quad (8)$$

где PID_p , PID_r — два экземпляра `PIDRegulator` с одинаковыми настраиваемыми K_p , K_i , K_d .

5.3 Сценарий `linear_chain2`: концевые точки, NED→корпус, закон Anatoliy

Концевые дроны. Для подлёта к целевой точке p^* в общем NED используются два регулятора `PIDRegulator` (`pitch_pid` с $K_p = \text{ENDPOINT_X_KP}$, `roll_pid` с $K_p = \text{ENDPOINT_Y_KP}$) и ошибки

$$e_x = x^* - x, \quad e_y = y^* - y \quad (9)$$

после опционального обнуления малых отклонений (`ERROR_DEADBAND_M`). Вектор $(e_x, e_y)^\top$ поворачивается из NED в горизонтальный базис корпуса “вперёд–вправо” с помощью угла рыскания ψ из `ATTITUDE` (с множителем `YAW_SIGN`):

$$e_{\text{fwd}} = e_x \cos \psi + e_y \sin \psi, \quad e_{\text{right}} = -e_x \sin \psi + e_y \cos \psi. \quad (10)$$

Далее $u_{\text{pitch}} \propto \text{PID}(e_{\text{fwd}})$, $u_{\text{roll}} \propto \text{PID}(e_{\text{right}})$ с тем же смыслом нейтрали u_{neut} и ограничениями `ENDPOINT_OUTPUT_LIMIT`.

Внутренние дроны: расстояния и функции g , ξ , σ . Пусть $u_{\text{seg}} = (u_x, u_y)^\top$ — единичный вектор вдоль отрезка между левым и правым якорем в плоскости Oxy NED, $n_{\text{seg}} = (-u_y, u_x)^\top$ — перпендикуляр “влево”. Для каждого соседа j с относительным вектором $r_j = p_j - p$ в плоскости Oxy вводятся координаты в базисе отрезка: $a_j = r_j \cdot u_{\text{seg}}$, $c_j = r_j \cdot n_{\text{seg}}$. По спискам $\{a_j\}$, $\{c_j\}$ для четырёх квадрантов вычисляются минимальные положительные расстояния d_a^+ , d_c^+ , d_a^- , d_c^- (аналог четырёх направлений в `corridor_sweeping`) и флаги наличия соседа f_a^+ , f_c^+ , f_a^- , f_c^- .

Функция виртуального зазора (аналог параметра w в коридорной модели):

$$g(d, w, f) = \begin{cases} d, & f = \text{истина (сосед есть)}, \\ w, & f = \text{ложь}. \end{cases} \quad (11)$$

Нелинейность $\xi(g) = \tanh(g)$. При нулевых составляющих скорости вдоль/поперёк отрезка ($v_{\text{along}} = v_{\text{cross}} = 0$ в коде для устойчивости к шуму) величины

$$\sigma_a = v_{\text{along}} - \xi(g(d_a^+, 0, f_a^+)) + \xi(g(d_a^-, 0, f_a^-)), \quad (12)$$

$$\sigma_c = v_{\text{cross}} - \xi(g(d_c^+, w_{\text{cross}}, f_c^+)) + \xi(g(d_c^-, w_{\text{cross}}, f_c^-)), \quad (13)$$

где $w_{\text{cross}} = \text{W_CROSS_M}$. Для геометрической части вводятся: p_A — положение левого якоря; p_i — положение текущего внутреннего дрона; $s_i = (p_i - p_A)^\top u_{\text{seg}}$ — координата дрона вдоль отрезка; s_{i-} и s_{i+} — координаты ближайших соседей слева и справа по отсортированной проекции на отрезок. Тогда ошибки выравнивания можно записать как

$$e_{\text{along}} = \frac{s_{i-} + s_{i+}}{2} - s_i, \quad e_{\text{cross}} = (p_i - p_A)^\top n_{\text{seg}}. \quad (14)$$

Вектор “комбинированной” цели в NED:

$$\begin{pmatrix} g_x \\ g_y \end{pmatrix} = k_{\text{along}} e_{\text{along}} u_{\text{seg}} - k_{\text{cross}} e_{\text{cross}} n_{\text{seg}}, \quad \begin{pmatrix} a_x \\ a_y \end{pmatrix} = \sigma_a u_{\text{seg}} + \sigma_c n_{\text{seg}}, \quad (15)$$

$$\begin{pmatrix} c_x \\ c_y \end{pmatrix} = \begin{pmatrix} g_x \\ g_y \end{pmatrix} + w_A \begin{pmatrix} a_x \\ a_y \end{pmatrix}, \quad (16)$$

где $k_{\text{along}} = \text{SPACING_ALONG_K}$, $k_{\text{cross}} = \text{SPACING_CROSS_K}$, $w_A = \text{ANATOLIY_COMB_WEIGHT}$. Далее $(c_x, c_y)^\top$ переводится в базис корпуса (fwd, right), вычитается демпфирование по проекциям скорости NED на корпус с коэффициентом `INTERNAL_BODY_V_DAMP`, и по ошибкам формируются приращения PWM пропорционально `INTERNAL_RC_KP` с ограничениями и наклоном (`_slew_int`).

6 Физика и динамика полёта: ответственность SITL и Webots

6.1 Роль кода Python в данном репозитории

Код Python в `Drone_Swarm_Simulator_v2` не реализует уравнения жёсткого тела мультикоптера и не выполняет численное интегрирование положения/скорости аппарата. Кинематика и динамика носителя рассчитываются в **ArduPilot SITL** (встроенная модель мультикоптера) либо в **Webots** с передачей состояния обратно в SITL через интерфейс внешней FDM-модели.

6.2 Мультикоптер в SITL: иерархия классов и силы

В файле `SIM_Multicopter.cpp` класс `MultiCopter` наследует `Aircraft`; метод `calculate_forces` вызывает расчёт сил и моментов рамы (`frame->calculate_forces`), формируя вектор углового ускорения в связанной системе $\dot{\omega}$ (в коде передаётся как `rot_accel`) и линейное ускорение в связанной системе `accel_body`. Затем `update_dynamics(rot_accel)` интегрирует состояние (см. ниже).

Какую именно модель использует SITL. Для мультикоптера в режиме SITL-only используется встроенная модель `SITL::MultiCopter` с типом рамы `FRAME_CLASS = 1` (*multicopter*) и `FRAME_TYPE = 1` (*X-class*) из файла `iris.parm`. По своей физической постановке это модель **жёсткого тела с шестью степенями свободы (6-DOF)**: три поступательных степени свободы описывают движение центра масс в земной системе координат, а три вращательных степени свободы — ориентацию и угловое движение корпуса. Силы и моменты формируются из вкладов отдельных роторов, причём для каждого ротора учитываются команда двигателя, напряжение батареи, плотность воздуха, входная скорость потока на винт, реактивный момент и аэродинамическое сопротивление. Следовательно, SITL в данном проекте использует не упрощённую кинематическую модель точки, а полноценную жёсткотельную модель мультикоптера с расчётом тяги, моментов и последующим интегрированием состояния.

Используемая модель коптера и автопилотной платформы. Если понимать “модель коптера” как тип виртуального летательного аппарата (по аналогии с *Mavic*, *Crazyflie* и т.п.), то в данном проекте используется не специализированная потребительская платформа такого класса, а стандартная мультикоптерная конфигурация **ArduPilot ArduCopter**. Лаунчер запускает `sim_vehicle.py` с ключом `-v ArduCopter`, а в самом ArduPilot для ArduCopter по умолчанию выбирается рама `quad`. Дополнительно файл параметров проекта `iris.parm` задаёт `FRAME_CLASS = 1` (*multicopter*) и `FRAME_TYPE = 1` (*X-class*). Следовательно, наиболее точное описание аппаратной модели в рамках данного стенда — это **четырёхроторный мультикоптер X-конфигурации под управлением ArduCopter**; в терминологии практического моделирования такую платформу можно считать *iris-like* конфигурацией ArduPilot, но не моделью DJI Mavic и не моделью Crazyflie.

Над этой динамической моделью работает штатный программный стек **ArduPilot ArduCopter**, включающий обработку режимов полёта, оценивание состояния и внутренние контуры стабилизации. В рассматриваемом проекте сценарии взаимодействию

ют с этой прошивкой через MAVLink-команды смены режима и через RC_OVERRIDE; в частности, используются режимы GUIDED, POS_HOLD, BRAKE и LAND. По логике ArduCopter это каскадная система управления: внешний контур по положению формирует требуемую скорость, контур скорости формирует требуемое ускорение или эквивалентные целевые углы наклона, далее контур ориентации формирует требуемые угловые скорости, а внутренний скоростной контур по крену, тангажу и рысканию преобразует эти требования в моторные команды. Поэтому управляющий код Python в проекте задаёт лишь внешний уровень целеуказания, тогда как непосредственная стабилизация коптера реализуется прошивкой ArduCopter.

6.3 Дискретное обновление в Aircraft::update_dynamics

В SIM_Aircraft.cpp используется шаг по времени Δt , вычисляемый из frame_time_us. Обозначим: ω — вектор угловой скорости в связанной системе (в коде накапливается в переменной gyro как угловая скорость); R — матрица направляющих косинусов (переход из связанной системы в земную, dcm); a_b — ускорение в связанной системе до учёта показаний акселерометра (accel_body после сил двигателей); g — ускорение свободного падения, $g = (0, 0, G)^\top$ в земной системе с осью z вниз (GRAVITY_MSS); v — скорость в земной системе (velocity_ef); p — положение (position).

Последовательность в явном виде в коде:

$$\omega \leftarrow \omega + \dot{\omega} \Delta t, \quad (17)$$

$$R \leftarrow R \cdot \Delta R(\omega \Delta t), \quad (18)$$

$$a_e = R a_b + g, \quad (19)$$

$$v \leftarrow v + a_e \Delta t, \quad (20)$$

$$p \leftarrow p + v \Delta t, \quad (21)$$

с последующим пересчётом accel_body как «кинематическое ускорение + гравитация», видимое акселерометром (строки с accel_body = dcm.transposed() * (...)). Здесь $\Delta R(\omega \Delta t)$ — малый поворот за шаг Δt , соответствующий вызову dcm.rotate(gyro * delta_time).

Непрерывный аналог удобно записать как систему Ньютона–Эйлера. Пусть m — масса аппарата; $f_b \in \mathbb{R}^3$ — результирующая сила в связанных осях; $M_b \in \mathbb{R}^3$ — результирующий момент в связанных осях; $J \in \mathbb{R}^{3 \times 3}$ — тензор инерции; $\omega \in \mathbb{R}^3$ — угловая скорость; $R \in SO(3)$ — матрица поворота из связанной системы в земную; $p \in \mathbb{R}^3$ — положение; $v \in \mathbb{R}^3$ — линейная скорость; $g \in \mathbb{R}^3$ — вектор ускорения свободного падения. Тогда

$$\dot{p} = v, \quad (22)$$

$$m\dot{v} = R f_b + m g, \quad (23)$$

$$J\dot{\omega} = M_b - \omega \times (J\omega), \quad (24)$$

$$\dot{R} = R S(\omega), \quad (25)$$

где $S(\omega)$ — кососимметричная матрица, задающая оператор векторного произведения: $S(\omega)x = \omega \times x$ для любого $x \in \mathbb{R}^3$. В коде детали формирования f_b и M_b скрыты в frame->calculate_forces, а update_dynamics реализует их численное интегрирование явным шагом.

Поэтому модель полёта удобно характеризовать как **шестистепенную модель жёсткого тела** (6-DOF): три поступательных и три вращательных степени свободы. По порядку уравнений это две подсистемы второго порядка (поступательная и вращательная), а в форме пространства состояний первого порядка — **12-мерная модель**, если вектор состояния взять в виде $x = (p^\top, \eta^\top, v^\top, \omega^\top)^\top$, где η — тройка параметров ориентации (например, углы Эйлера).

Формулировка для дипломного текста. Таким образом, полётная динамика в рассматриваемом стенде моделируется встроенной подсистемой ArduPilot SITL MultiCopter. Эта подсистема реализует пространственную жёсткотельную модель мультикоптера типа “X” с шестью степенями свободы. С математической точки зрения модель включает поступательное движение центра масс и вращательное движение корпуса под действием суммарной тяги роторов, реактивных моментов и аэродинамического сопротивления. По порядку исходных дифференциальных уравнений это связанная система второго порядка для линейного и углового движения, а после приведения к нормальной форме Коши она эквивалентна системе **12 дифференциальных уравнений первого порядка** по координатам, скоростям, ориентации и угловым скоростям летательного аппарата.

6.4 Параметр SCHED_LOOP_RATE

В файле `config/iris.parm` задано `SCHED_LOOP_RATE 300`: частота основного цикла планировщика автопилота в симуляции порядка 300 Гц. Обмен сообщениями MAVLink и логика координации роя в Python работают на существенно более низких внешних частотах (десятки герц), т. е. это внешние контуры относительно быстрого внутреннего контура автопилота.

6.5 Режим Webots (SIM_Webots_Python.cpp)

Класс `WebotsPython` получает от внешней FDM (`Webots`) пакет `fdm_packet`: линейное ускорение IMU в связанных осях, угловую скорость, углы ориентации, вектор скорости и положение в земной системе; эти величины записываются в `accel_body`, `gyro`, `dcm`, `velocity_ef`, `position`. Управляющие входы передаются в `Webots` как нормированные скорости моторов (`send_servos`). Таким образом, в режиме `Webots` интегрирование динамики выполняет движок `Webots`; SITL использует возвращаемое состояние для согласования с остальной цепочкой автопилота.

7 Заключение

Проект `Drone_Swarm_Simulator_v2` связывает несколько автономных SITL-автопилотов с одним координирующим процессом Python: телеметрия позиции и ориентации запрашивается на 50 Гц, цикл обмена координатами по умолчанию согласован с $f_{\text{exch}} = 50$ Гц, поддержание RC — каждые 0,2 с, визуализатор опционально получает JSON по UDP на `127.0.0.1:15551`. Динамика носителя определяется ArduPilot SITL (класс `MultiCopter` и интегратор `Aircraft::update_dynamics`) либо `Webots` через `WebotsPython`.